Asynchronous programming

Prepared by Moustafa Dawood
Supervised By **Mohamed A. EL-adawy**

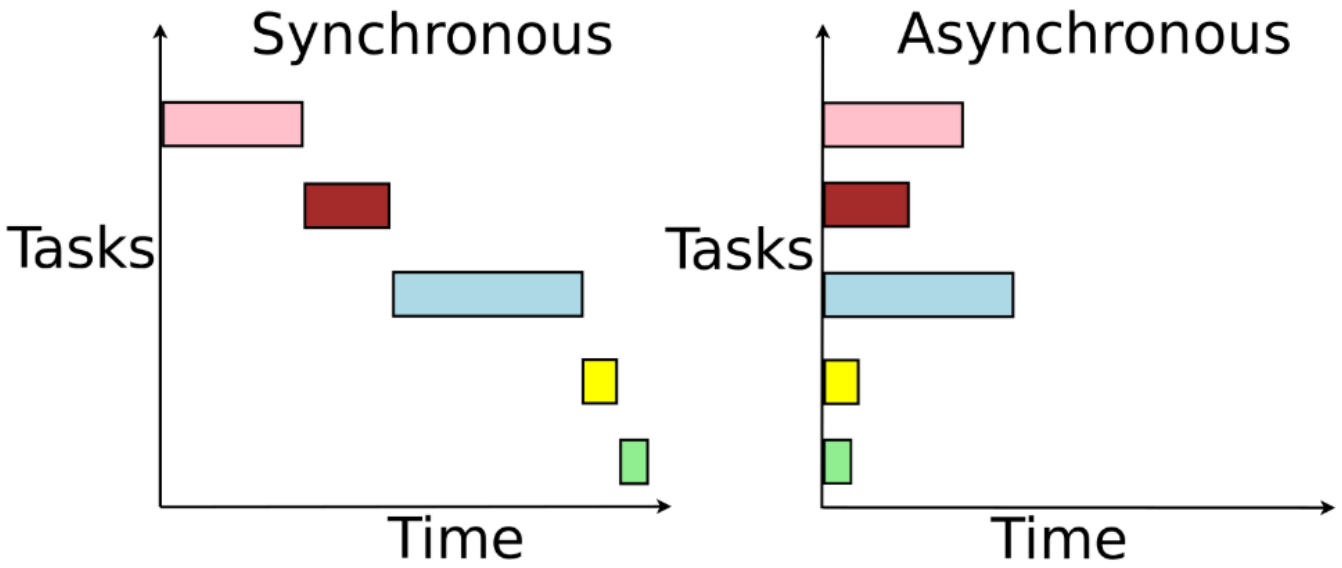
Introduction: From Synchronous to Asynchronous Programming

For many years, most programs were written using synchronous programming.

In synchronous execution:

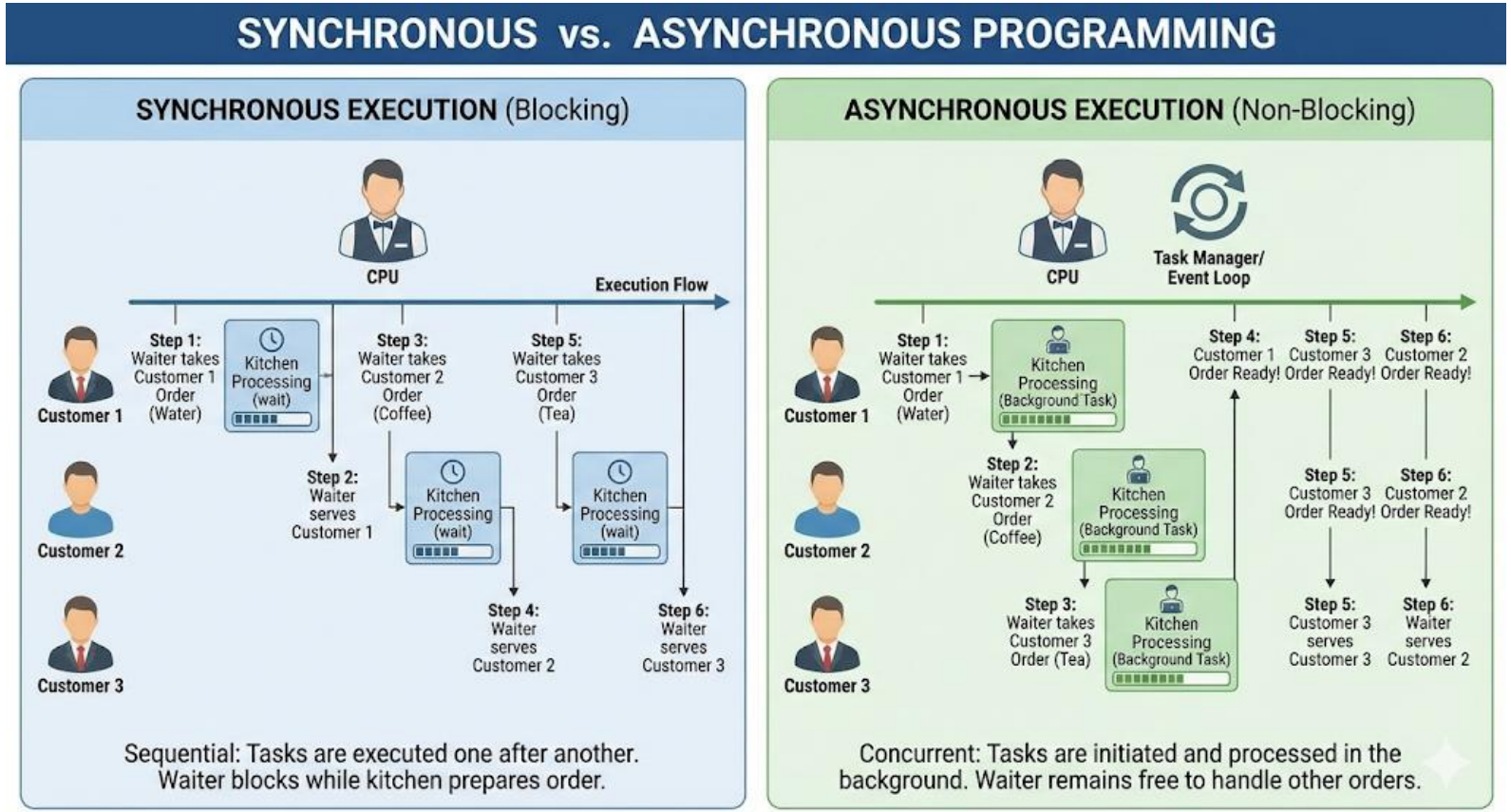
- Code runs line by line.
- Each instruction must finish before the next begins.
- If a function waits, the entire program waits

- **Asynchronous programming is a concurrency model that allows:**
- Multiple tasks to make progress over time
- Tasks to pause when waiting
- Other tasks to run during that wait
- Instead of blocking the whole program, only the waiting task is suspended.
- The rest of the system continues executing.



Before we talk about Python, we need to talk about execution models. Imagine a testbench as a restaurant. We have one CPU core (the Waiter) and three tasks that need to run (the Customers), each taking a different amount of time to process.

How do we execute these tasks? **We have three options**:



Way 1: Synchronous Execution (The Blocking Waiter)

- The Analogy: The Waiter takes Customer 1's order, walks to the kitchen, and stands there staring at the chef until the food is done. Customers 2 and 3 sit there waiting, totally ignored.

- The SV/UVM Reality: This is standard, top-to-bottom code. If you call a blocking task in SystemVerilog without a fork, the entire simulation thread freezes at that line. Nothing else in your testbench can happen until that task finishes. It is safe but too slow .

Way 2: Multithreading (The Parallel Waiters)

- The SV/UVM Reality: This is how actual Verilog hardware works! Multiple always blocks execute in parallel. However, we cannot do this in Python testbenches. Why? Because the Verilog simulator (Questa) communicates with Python via a C-interface (VPI/DPI-C). The IEEE standard dictates this interface is strictly single-threaded. If three true OS threads try to inject data into the Verilog simulator at the same exact picosecond, the simulator will instantly crash (Segfault) and that's **called Multithreading**

Way 3: Asynchronous Execution (The Concurrent Waiter)

- The Analogy: We only have one Waiter (one thread), but they are incredibly smart. They take Customer 1's order, pass it to the kitchen, and instantly turn to Customer 2 to take their order. The Waiter never stands idle. When the kitchen rings the bell (an event), the Waiter pauses what they are doing to deliver the food.

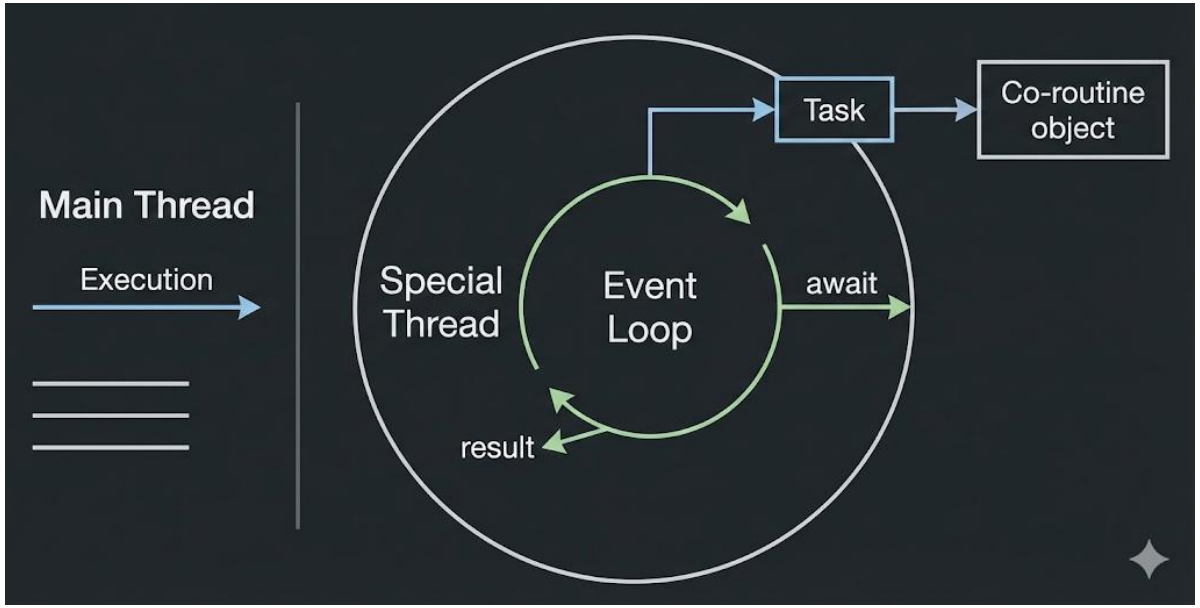
- The SV/UVM Reality: This is exactly how the Verilog Event Scheduler works under the hood. It only evaluates one thing at a microsecond, but it rapidly switches between active always blocks, #delays, and @(posedge clk) events

The Main Thread

When you run a standard Python script, the interpreter runs on the Main Thread. It reads your code line-by-line, in strict sequence

The Special Thread (The Event Loop)

To fix this blocking issue in hardware verification, the asyncio library (which Cocotb uses) creates a "Special Thread" known as the **Event Loop**. Instead of the Main Thread executing code line-by-line blindly, it hands the code over to the Event Loop. The Event Loop acts as a master scheduler



Task Scheduling

Imagine Tasks 1, 2, and 3 are handed to the Event Loop (for example, a Driver, a Monitor, and a Watchdog timer). The Event Loop uses **cooperative multitasking**, meaning it cannot forcefully interrupt a running task. It relies entirely on each task explicitly handing control back to the engine (using the **await** keyword) whenever it needs to wait, allowing other components to take their turn

Returning Result

Because the Event Loop is constantly switching to whichever task is "awake" and ready to do math, tasks that require less hardware waiting will naturally finish and return their result first. The heavy, slow tasks stay safely in the background, waking up only when their specific await condition is met, completely preventing the Main Thread from freezing

Coroutine Object

Think of a coroutine object simply as a wrapper around your function's begin and end. It internally tracks the exact execution status of your code—whether it is just starting, currently pending on an await condition, or has finally returned a result

Before we write our concurrent components, let's look at what happens if we write a standard Python script. This code represents our Waiter taking three orders: Coffee (3 seconds), Tea (2 seconds), and Water (1 second)

#Terminal

```
(venv) λ py sync_code.py
start coffee request ...
finished coffee request ...
start tea request ...
finished tea request ...
start water request ...
finished water request ...
 code time finished: 6.0
```

As the first start it blocks the another till finish and its takes their total time

```python
start_code_time = time.time()

def waiter(customer_request, duration):
    print(f"start {customer_request} request ...")
    time.sleep(duration)

    print(f"finished {customer_request} request ...")

def main():
    # coffee
    waiter("coffee", 3)
    # tea
    waiter("tea", 2)
    # water
    waiter("water", 1)


if __name__ == '__main__':
    main()
    print(f" code time finished: {round(time.time() - start_code_time, 1)}")
```

Modifications for asynchronous code

1)Make Coroutine Object by using **async** keyword for functions and as the code asynchronous it must wait so we Must use **await** keyword

Running the code without await key word will get and error

"Coroutine never awaited"

```
(venv) λ py async_code.py
E:\crawler\asyncio_tutorial\async_code.py:14: RuntimeWarning: coroutine 'waiter' was never awaited
  waiter("coffee", 3)
RuntimeWarning: Enable tracemalloc to get the object allocation traceback
E:\crawler\asyncio_tutorial\async_code.py:15: RuntimeWarning: coroutine 'waiter' was never awaited
  waiter("tea", 2)
RuntimeWarning: Enable tracemalloc to get the object allocation traceback
E:\crawler\asyncio_tutorial\async_code.py:16: RuntimeWarning: coroutine 'waiter' was never awaited
  waiter("water", 1)
RuntimeWarning: Enable tracemalloc to get the object allocation traceback
 code time finished: 0.0
```

So we need to add await key word for Coroutine Object and also if
Coroutine is calling another Coroutine

```python
import time

start_code_time = time.time()

async def waiter(customer_request, duration):
    print(f"start {customer_request} request ...")
    asyncio.sleep(duration)

    print(f"finished {customer_request} request ...")


async def main():
    waiter("coffee", 3)
    waiter("tea", 2)
    waiter("water", 1)

if __name__ == '__main__':
    asyncio.run(main())
    print(f" code time finished: {round(time.time() - start_code_time, 1)}")
```

Although after fixing the **await** key word we got that

Both async code and sync code take **same time** even if they are different techniques .. What's Missing??

```
(venv) λ py async_code.py
start coffee request ...
finished coffee request ...
start tea request ...
finished tea request ...
start water request ...
finished water request ...
 code time finished: 6.0

E:\crawler\asyncio_tutorial
(venv) λ py sync_code.py
start coffee request ...
finished coffee request ...
start tea request ...
finished tea request ...
start water request ...
finished water request ...
 code time finished: 6.0
```

```python
import time

start_code_time = time.time()

async def waiter(customer_request, duration):
    print(f"start {customer_request} request ...")
    await asyncio.sleep(duration)

    print(f"finished {customer_request} request ...")


async def main():
    await waiter("coffee", 3)
    await waiter("tea", 2)
    await waiter("water", 1)

if __name__ == '__main__':
    asyncio.run(main())
    print(f" code time finished: {round(time.time() - start_code_time, 1)}")
```

Just adding the async keyword doesn't magically make things parallel. We need to explicitly explain how to utilize the scheduler.

To fix our 6-second execution problem, we need a way to tell the Event Loop: *"Here are three orders. Start working on them in the background immediately."* In Python, we do this by **wrapping our Coroutine Object inside a Task using asyncio.create_task().** (In Cocotb hardware testbenches, the exact same command is called **cocotb.start_soon**())

#output after modification

```
(venv) λ py async_code.py
start coffee request ...
start tea request ...
start water request ...
finished water request ...
finished tea request ...
finished coffee request ...
 code time finished: 3.0
```

```
async def main():
    task_1 = asyncio.create_task(waiter("coffee", 3))
    task_2 = asyncio.create_task(waiter("tea", 2))
    task_3 = asyncio.create_task(waiter("water", 1))

    await task_1
    await task_2
    await task_3
```

We Can Further enhance that code by using **asyncio.gather()** function we discuss it Later.

We've just looked at Coroutines, the Event Loop, and Task scheduling. But it is crucial to understand that Asynchronous Programming is a huge domain.

The Fundamentals

- Foundations (Concurrency Theory)
- Blocking vs Non-Blocking
- Threads vs Processes vs Async
- Event Loop Internals
- Coroutines (async / await)
- Tasks & Scheduling
- Futures
- asyncio Core APIs
- Queues
- Synchronization Primitives
- Timeouts & Cancellation
- Error Handling
- Performance & Pitfalls
- Real Architectures (Producer–Consumer,)
- Async in Hardware Verification (Cocotb mapping)
- Advanced Topics (Backpressure, Task Groups, Debugging)

Advanced OOP (Classes, Instances, and State)

In standard scripting, you write functions that run from top to bottom. In verification, you are building a system of independent "actors" (a Driver, a Monitor, a Checker) that all live at the same time and talk to each other. To do this, we use Classes.

When you create a new object, Python automatically runs a special setup function called __init__ (short for initialize)

```
class Transaction:
    # This runs the exact moment you type: my_txn = Transaction(1, 55)
    def __init__(self, tid, data):
        self.tid = tid
        self.data = data
```

__init__ and self == The SV **new()** function and **this** handle

- UVM/SV: When you create a class in SystemVerilog, you write a function new(string name, uvm_component parent);. Inside that function, if an argument has the same name as a class property, you use this.data = data; to point to the specific instance's variable.
- Python: __init__ is exactly the new() function. And self is exactly the this handle. Python just forces you to write self explicitly every time

Exception Handling

In standard SystemVerilog, when something goes wrong, you use macros like **`uvm_error** to log a mismatch, or **`uvm_fatal** to instantly kill the simulation.

Python handles errors completely differently. It uses an Exception System. When Python encounters an error (like dividing by zero, or a simulator timeout), it creates an Exception object and "throws" it. If your code does not explicitly "catch" that exception, it bubble all the way up to the Main Thread, crashes the Python interpreter, and violently terminates your simulation.

Here is how we control that behavior using **try, except, raise, and finally**

1-try and except

- The Concept: We "try" to execute risky code. If a specific error happens, we "catch" it and handle it gracefully instead of letting the simulation crash
-

```python
def calculate_ratio(hits, total_attempts):
    try:
        # Risky code: What if total_attempts is 0?
        ratio = hits / total_attempts
        print(f"Success ratio: {ratio}")

    except ZeroDivisionError:
        # We caught the crash! Handle it gracefully.
        print("UVM_WARNING: Tried to divide by zero. Defaulting ratio to 0.")
        ratio = 0
```

2-raise (The UVM Fatal) --we want to kill the test before it even starts driving pins.

```
def set_clock_speed(frequency):
    if frequency <= 0:
        # We manually create an error and throw it.
        # This acts exactly like a uvm_fatal.
        raise ValueError("UVM_FATAL: Clock frequency must be greater than zero!")

    print(f"Clock successfully set to {frequency} MHz")
```

3-finally (Guaranteed Cleanup)

- The Concept: A block of code that is 100% guaranteed to run, regardless of whether the try block succeeded or completely crashed.
- The SV/UVM Mapping: Ensuring you call **phase.drop_objection(this)** at the end of a sequence so the test doesn't hang forever

```
def run_main_phase():
    print("UVM_INFO: Raising objection...")

    try:
        print("Running test sequence...")
        # Imagine a fatal error gets triggered here
        raise RuntimeError("The DUT just locked up!")

    except RuntimeError as e:
        print(f"UVM_ERROR: Caught a simulator issue -> {e}")

    finally:
        # GUARANTEED EXECUTION:
        # This will print even if the 'except' block didn't exist!
        print("UVM_INFO: Dropping objection...")
```

Decorators (The @ Syntax)

In standard software, a Decorator is a function that takes *another* function, wraps some extra code around it, and returns the enhanced version. It alters behavior without changing the original source code

- UVM/SV: When you write a new test or component in UVM, you put **`uvm_component_utils(my_class)** at the top. This macro **secretly injects a bunch of background** code that registers your class with the UVM Factory so the UVM Phasing engine knows it exists.
- Python: Decorators do the exact same thing without macros. You put **@cocotb.test()** above your function to register it with the Cocotb simulation engine

WHEN TO USE IT: You will use @cocotb.test() above every top-level test

```python
# The Decorator intercepts your function before the simulation starts
@cocotb.test()
async def my_alsu_test(dut):
    # Test code here
    pass
```

Generators

- Standard Function (return): Think of this like watching a movie in a theater. You start it, it plays continuously to the end, the credits roll, and you are kicked out. All local variables are erased. If you want to run it again, you must start from the very beginning.
- Generator Function (yield): Think of this like reading a book. You read a chapter, place a bookmark in the page (yield), and put the book down. Your exact place is saved in memory. Tomorrow, you open it and resume reading from the *exact sentence* where you stopped.
- **SystemVerilog:** When a UVM sequence or SV task hits @(posedge clk) or #10ns, the task yields control back to the Verilog Event Scheduler. The local variables in that task don't die; the thread is simply frozen. When the clock ticks, the scheduler wakes the thread back up.
- **Python:** Standard Python execution doesn't have a concept of hardware time or pausing. Generators (and the yield keyword) were the clever software hack invented to mimic Verilog's frozen threads

```python
@cocotb.test()
def old_school_test(dut):
    print("Starting test...")
    # The function PAUSES here and hands the Timer object to the Cocotb engine
    yield Timer(10, "ns")
    # The simulator engine wakes the function up exactly 10ns later!
    print("10 ns have passed!")
```

They renamed yield to **await**, and they required you to label the function with **async def**.

```python
from cocotb.triggers import RisingEdge, Timer

# 1. We declare this as a coroutine (a modern generator) using 'async def'
async def monitor_dds_calibration(dut):
    print("Initiating DDS calibration...")
    dut.start_calib.value = 1

    # 2. We place a bookmark and PAUSE Python until the next clock edge
    await RisingEdge(dut.clk)
    dut.start_calib.value = 0

    # 3. We place another bookmark, yielding control back to the simulator
    # until the hardware pulls the 'calib_done' signal high.
    while dut.calib_done.value == 0:
        await RisingEdge(dut.clk)

    print("DDS Calibration complete! Resuming Python execution.")
```

Typing await is literally just placing a bookmark in your Python code, yielding control to the Cocotb C++ simulator, and waiting for the simulator to wake it back up

Async Fundamentals

The Event Loop & Scheduling

To write asynchronous code, you have to understand the engine that runs it. In Python, this engine is called the Event Loop.

In standard Python, your script starts at line 1, runs to line 100, and dies. In Async Python, your script starts an infinite while True: loop. This loop sits in the background, keeping track of all your functions, waking them up when they are ready, and pausing them when they are waiting

In pure Python, you have to create and start the Event Loop yourself. We use the **built-in asyncio library**. Notice how the Event Loop handles the await command by switching back and forth between task_1 and task_2

```python
import asyncio
import time

async def fetch_data(id, delay):
    print(f"Task {id}: Requesting data from server...")
    # The 'await' tells the Event Loop: "I am paused. Go run another task!"
    await asyncio.sleep(delay)
    print(f"Task {id}: Data received!")

async def main_loop_setup():
    # We hand two tasks to the Event Loop to manage concurrently
    task_1 = asyncio.create_task(fetch_data(1, 2.0)) # Takes 2 seconds
    task_2 = asyncio.create_task(fetch_data(2, 1.0)) # Takes 1 second

    # We tell the main program to wait until the Event Loop finishes both
    await task_1
    await task_2

# This is the command that actually boots up the Event Loop engine!
asyncio.run(main_loop_setup())

# OUTPUT:
# Task 1: Requesting data from server...
# Task 2: Requesting data from server...
# Task 2: Data received!  <-- Finished first because the Event Loop didn't block!
# Task 1: Data received!
```

Cocotb Scope

Cocotb completely deletes the standard Python asyncio Event Loop, Instead, Cocotb hijacks the Python thread and hands control over to Questa/Modelsim's internal Verilog Event Loop (via the VPI interface)

Notice we didn't type asyncio.run(). The **@cocotb.test() decorator automatically binds this function to the QuestaSim** event loop when you type make in the terminal

```python
import cocotb
from cocotb.triggers import Timer

@cocotb.test()
async def hardware_test(dut):
    print("Test started at 0 ns")

    # We DO NOT use asyncio.sleep(). It would break the simulator!
    # Instead, we yield control to QuestaSim's Event Loop.
    await Timer(10, units="ns")

    # QuestaSim advances simulated hardware time to 10ns,
    # then wakes this Python function back up.
    print("Test resumed at 10 ns")
```

Coroutines and Task Scheduling

In Python, a standard function executes in zero simulation time. A Coroutine is a special function that has the ability to pause itself, hand control back to the Event Loop, and wake up later

Standard Function (def) ------Coroutine (async def)+ (await)

- UVM/SV: A function in SystemVerilog executes in zero time and cannot contain delays. A task can consume time (using #10 or @posedge clk).
- Python: def == SV function
- async def == SV task
- await == #delay or @(event)

```
import asyncio

# The 'async def' tells Python: "This function is allowed to pause"
async def bake_cake():
    print("Cake in oven...")
    await asyncio.sleep(2) # We pause here and yield control
    print("Cake is done!")
```

Cocotb Scope

will use this in every single component you write

```
from cocotb.triggers import RisingEdge

# We define our hardware task
async def wait_for_clock(dut):
    print("Waiting for clock edge...")
    # We yield control back to the QuestaSim scheduler!
    await RisingEdge(dut.clk)
    print("Clock ticked!")
```

WHEN TO USE IT: Use async def for *any* method in your testbench that needs to consume simulation time (like your Driver's run() method or Monitor's run() method). If a function contains an await statement inside it, it *must* be defined with async def

Task Scheduling (start_soon vs await)

- UVM/SV: If you call my_task(); normally, the code blocks until the task finishes. If you want that task to run in the background as an independent thread, you wrap it in a fork my_task**(); join_none block**.
- Python: **await my_coroutine()** is a normal blocking task call. **cocotb.start_soon(my_coroutine()) is the exact equivalent of fork ... join_none**

WHEN TO USE IT: Use cocotb.start_soon() inside your top-level @cocotb.test() function to launch your Drivers, Monitors, and Checkers. This spins them up as independent, concurrent hardware threads so they can all talk to each other simultaneously.

```python
import asyncio

async def background_worker():
    await asyncio.sleep(5)
    print("Background work finished!")

async def main():
    # This acts like fork ... join_none
    bg_task = asyncio.create_task(background_worker())

    # We DO NOT wait 5 seconds! This prints instantly.
    print("Worker launched! I am moving on to other things.")
```

Queues & Synchronization

Now that we know how to launch multiple background tasks (like a Driver and a Checker) running at the exact same time, we run into a massive problem: How do they safely talk to each other?

Queues

The SV/UVM Mapping (Mailboxes and TLM FIFOs)

- UVM/SV: In SystemVerilog, using a mailbox to safely pass handles between threads. In UVM, you use a uvm_tlm_fifo connected to analysis ports so the Monitor can push sequence items into the Scoreboard's memory.
- Python: asyncio.Queue (or cocotb.queue.Queue for hardware) is the exact equivalent of a TLM FIFO.

```python
import asyncio

async def producer(queue):
    for i in range(3):
        print(f"[WAITER] Pushing order {i} to the rail.")
        await queue.put(i)       # Waiter pushes data IN
        await asyncio.sleep(1)

async def consumer(queue):
    while True:
        order = await queue.get() # Chef waits for data OUT
        print(f"[CHEF] Cooking order {order}")
        queue.task_done()

async def main():
    q = asyncio.Queue()
    # Launch them concurrently!
    asyncio.create_task(producer(q))
    asyncio.create_task(consumer(q))
    await asyncio.sleep(4)
```

Or As simple as

Push data

```python
# Inside monitor.py
data = int(self.dut.data_out.value)
await self.out_queue.put(data) # Pushing to the TLM FIFO
```

Wait for it

```python
# Inside checker.py
result = await self.out_queue.get() # Pulling from the TLM FIFO
```

WHEN TO USE IT: Use Queues whenever data crosses boundaries between your Verification Components. Monitor -> Scoreboard, or Sequencer -> Driver. It perfectly buffers data so fast components don't crash slow components

Synchronization

In a UVM testbench, concurrent components constantly need to talk to each other. For example, Monitor shouldn't start capturing packets until the Reset Sequence has finished.

In SystemVerilog, we solve this using the event data type. we trigger it with the -> operator, and we wait for it using the @ operator.

In asynchronous Python and Cocotb, the exact equivalent is cocotb.triggers.Event. Here is how the API maps directly to SystemVerilog

The Cocotb Event API:

- Set(): The Trigger.
  - *SV Mapping:* -> my_event;
  - When a coroutine calls my_event.set(), it instantly wakes up every other coroutine that was waiting on this event.
- Wait(): The Blocker.
  - *SV Mapping:* @(my_event);
  - When a coroutine calls await my_event.wait(), it yields control to the Event Loop and freezes until another component calls set().
- Clear(): The Reset.
  - Unlike standard SV events which are momentary pulses, a Cocotb Event acts more like a uvm_event. Once it is set, it stays set. You use clear() to pull it back down so it can be triggered again.
- Is set().
  - *SV Mapping:* my_event.triggered
  - A non-blocking way to check the current status of the event without pausing execution.

```python
from cocotb.triggers import Event

# Create the shared event object
reset_done_event = Event(name="ResetDone")

async def reset_sequence(dut):
    print("Asserting reset...")
    dut.reset.value = 1
    await Timer(100, 'ns')
    dut.reset.value = 0
    print("Reset complete! Firing event.")

    # SV Equivalent: -> reset_done_event;
    reset_done_event.set()

async def packet_driver(dut):
    print("Driver waiting for reset to finish...")

    # SV Equivalent: @(reset_done_event);
    # This pauses the driver, yielding to the Event Loop
    await reset_done_event.wait()

    print("Event received! Driver starting to send packets.")
```

Timeouts

A timeout is a **wrapper placed around an await statement**. It tells the Event Loop: *"Pause my coroutine and wait for this event. But if it takes longer than X seconds, wake me up anyway and throw an error*

We wrap our slow task in asyncio.wait_for()

Cocotb Scope

Cocotb uses with_timeout() instead of asyncio.wait_for()

```python
import asyncio

async def fetch_database_record():
    print("[DB] Searching for record...")
    await asyncio.sleep(10) # Simulating a very slow database
    return "Data Found!"

async def main():
    try:
        print("Requesting data with a 3-second deadline...")
        # We wrap the await in a timeout
        result = await asyncio.wait_for(fetch_database_record(), timeout=3.0)
        print(f"Success: {result}")

    except asyncio.TimeoutError:
        # The Event Loop throws this error precisely at 3.0 seconds!
        print("CRITICAL: The database took too long! Aborting request.")

# Output:
# Requesting data with a 3-second deadline...
# [DB] Searching for record...
# CRITICAL: The database took too long! Aborting request.
```

Managing Multiple Tasks

Up until now, we've only launched tasks and let them run wild in the background (fork ... join_none). But what if we want to sync them all up? And more importantly, why are we using this weird asyncio Event Loop instead of just using standard Operating System Threads?

- UVM/SV: You use a standard fork ... join block. The main thread hits the fork, launches 5 child processes, and completely freezes at the join keyword until the absolute slowest child finishes.
- Python: **We use the asyncio.gather()** API, or in Cocotb, we can use Combine() or simply loop through and await a list of tasks

Cocotb Scope

If you have a multi-port Verilog DUT you want to drive data into all 4 ports simultaneously and wait for all 4 sequences to finish before ending the test

WHEN TO USE IT:
Use Combine or gather whenever you need to synchronize multiple independent hardware interfaces at a specific checkpoint in your simulation

```python
import cocotb
from cocotb.triggers import Combine

@cocotb.test()
async def multi_port_test(dut):
    # Launch 4 independent drivers in the background
    tasks = []
    for i in range(4):
        # start_soon returns the Task object (the Future)
        t = cocotb.start_soon(drive_port(dut, port_num=i))
        tasks.append(t)

    # Wait for ALL of them to finish before moving on
    # Combine takes a list of tasks/triggers and waits for all of them
    await Combine(*tasks)

    print("All 4 ports have finished transmitting. Test Complete!")
```

```python
import asyncio

async def driver_task(id, delay):
    print(f"[DRIVER {id}] Starting...")
    await asyncio.sleep(delay)
    print(f"[DRIVER {id}] Finished!")
    return f"Status_{id}_OK"

async def main():
    print("[MAIN] Launching 3 Drivers...")

    # We launch them all concurrently and ask asyncio to 'gather' the results
    # This acts EXACTLY like a fork ... join block!
    results = await asyncio.gather(
        driver_task(1, 2.0), # Finishes second
        driver_task(2, 1.0), # Finishes first
        driver_task(3, 3.0)  # Finishes last
    )

    # The main thread pauses until the 3-second task finishes.
    print(f"[MAIN] All drivers done! Results: {results}")

# Output:
# [MAIN] Launching 3 Drivers...
# [DRIVER 1] Starting...
# [DRIVER 2] Starting...
# [DRIVER 3] Starting...
# [DRIVER 2] Finished!
# [DRIVER 1] Finished!
# [DRIVER 3] Finished!
# [MAIN] All drivers done! Results: ['Status_1_OK', 'Status_2_OK', 'Status_3_OK']
```

Software Python (asyncio)	Hardware Python (cocotb)	SystemVerilog / UVM Equivalent	What it does
asyncio.sleep(10)	await Timer(10, 'ns')	#10ns;	Pauses the thread and advances time.
asyncio.create_task(coro)	cocotb.start_soon(coro)	fork ... join_none	Launches a background process.
asyncio.gather(*tasks)	await Combine(*tasks)	fork ... join	Blocks until ALL child tasks finish.
asyncio.wait(return_when=FIRST_COMPLETED)	await First(*tasks)	fork ... join_any	Blocks until the FIRST child task finishes.
asyncio.wait_for(coro, 10)	await with_timeout(coro, 10, 'ns')	fork ... join_any (with a #10ns watchdog)	Runs a task but kills it if it takes too long.
asyncio.Event()	cocotb.triggers.Event()	event / -> / @	Triggers multiple waiting threads simultaneously.
asyncio.Lock()	cocotb.triggers.Lock()	semaphore	Prevents multiple drivers from accessing a shared bus at the same time.

cocotb.triggers. These are specialized async objects that talk directly to the VPI/DPI-C interface of the simulator. They tell the simulator to advance simulated hardware time, wait for actual silicon signals, and then wake Python back up.

Cheat Sheet for Python asyncio

Why asyncio?

Asyncio provides coroutine-based concurrency suited to non-blocking socket I/O applications.

Coroutine

Import
```
import asyncio
```

Define a coroutine
```
async def custom_coroutine():
    # ...
```

Create coroutine object
```
coro = custom_coroutine()
```

Run coroutine as entry point
```
asyncio.run(main())
```

Suspend and run coroutine from a coroutine
```
await custom_coroutine()
```

Sleep a coroutine
```
await asyncio.sleep(1)
```

Async Comprehensions and Loops

Asynchronous comprehension
```
res = [r async for r in async_gen()]
```

Await comprehension
```
res = [r await a in awaitables]
```

Asynchronous for-loop
```
async for item in async_gen():
    print(item)
```

Task

A task schedules a coroutine to run independently.

Create and schedule task (high-level)
```
task = asyncio.create_task(coro)
```

Create and schedule task (low-level)
```
task = asyncio.ensure_future(coro)
```

Suspend and wait for a task to finish
```
await task
```

Get the current task
```
task = asyncio.current_task()
```

Get all running tasks
```
tasks = asyncio.all_tasks()
```

Get task result
```
value = task.result()
```

Get task unhandled exception
```
ex = task.exception()
```

Cancel a task result
```
was_canceled = task.cancel()
```

Check if the task is done (not running)
```
if task.done():
    # ...
```

Check if the task was canceled
```
if task.cancelled():
    # ...
```

Add done callback function
```
task.add_done_callback(handler)
```

Remove done callback function
```
task.remove_done_callback(handler)
```

Set and get task name
```
task.set_name('MyTask')
name = task.get_name()
```

Tasks

Operations on an awaitable, task, or tasks.

Wait for awaitable with a timeout
```
try:
    await asyncio.wait_for(tk,
timeout=1)
except asyncio.TimeoutError:
    # ...
```

Shield a task from cancelation
```
shielded = asyncio.shield(task)
```

Run blocking function in new thread
```
coro = asyncio.to_thread(myfunc)
```

Run coroutine in asyncio event loop
```
fut = run_coroutine_threadsafe(coro,
loop)
```

Run many awaitables as a group
```
await asyncio.gather(c1(), c2())
```

Wait for all tasks in a collection
```
done,pen = await asyncio.wait(tasks)
```

Wait for all tasks with a timeout in seconds
```
Try:
    done,pen = await
asyncio.wait(tasks, timeout=5)
except asyncio.TimeoutError:
    # ...
```

Wait for the first task in a collection
```
done,pen = await asyncio.wait(tasks,
return_when=FIRST_COMPLETED)
```

Wait for the first task to fail
```
done,pen = await asyncio.wait(tasks,
return_when=FIRST_EXCEPTION)
```

Get results in task completion order
```
for c in
asyncio.as_completed(tasks):
    result = await c
```

Non-blocking IO Subprocesses

Run command as subprocess
```
p = await
create_subprocess_exec('ls')
```

Run shell command as subprocess
```
p = await
create_subprocess_shell('ls')
```

Wait for subprocess to finish
```
await process.wait()
```

Read from subprocess
```
data = await process.communicate()
```

Read from subprocess
```
await
process.communicate(input=data)
```

Terminate a subprocess
```
process.terminate()
```

Non-blocking IO Streams

Open a client tcp connection
```
reader, writer = await
open_connection(google,com', 80)
```

Start a tcp server
```
server = await start_server(handle,
'127.0.0.1', 9876)
```

Read from socket
```
data = await reader.readline()
```

Write to socket
```
writer.write(data)
```

Drain socket until ready
```
await writer.drain()
```

Close socket connection
```
writer.close()
await writer.wait_closed()
```

Semaphores and Events, and Conditions

Semaphore, set num positions
```
semaphore = asyncio.Semaphore(10)
await semaphore.acquire()
# ...
semaphore.release()
```

Semaphore, context manager
```
async with semaphore:
    # ...
```

Create event, then set event
```
event = asyncio.Event()
event.set()
```

Check if event is set
```
if event.is_set():
    # ...
```

Wait for event to be set (blocking)
```
await event.wait()
```

Condition variable
```
condition = asyncio.Condition()
await condition.acquire()
# ...
condition.release()
```

Wait on condition to be notified (blocking)
```
async with condition:
    await condition.wait()
```

Wait on condition for expression (blocking)
```
async with condition:
    await condition.wait_for(check)
```

Notify any single thread waiting on condition
```
async with condition:
    condition.notify(n=1)
```

Notify all threads waiting on condition
```
async with condition:
    condition.notify_all()
```

Async Locks

Mutex lock
```
lock = asyncio.Lock()
await lock.acquire()
# ...
lock.release()
```

Mutex lock, context manager
```
async with lock:
    # ...
```

Queues

Via `Queue`, `LifoQueue`, `PriorityQueue`

Create queue
```
queue = asyncio.Queue()
```

Create queue with limited capacity
```
queue = asyncio.Queue(100)
```

Add item to queue (blocking, if limited)
```
await queue.put(item)
```

Retrieve item from queue (blocking)
```
item = await queue.get()
```

Check if queue is empty
```
if queue.empty():
    # ...
```

Check if queue is full
```
if queue.full():
    # ...
```

Get current capacity of queue
```
capacity = queue.qsize()
```

Mark unit of work complete
```
queue.task_done()
```

Wait for all units to be complete
```
await queue.join()
```

Async Generators and Iterators

Define asynchronous generator
```
async def async_generator():
    for i in range(10):
        await asyncio.sleep(1)
        yield i
```

Define asynchronous iterator
```
class AsyncIterator():
    def __init__(self):
        self.counter = 0
    def __aiter__(self):
        return self
    async def __anext__(self):
        if self.counter >= 10:
            raise StopAsyncIteration
        await asyncio.sleep(1)
        self.counter += 1
        return self.counter
```

Use asynchronous iterator
```
async for value in AsyncIterator():
    # ...
```

Async Context Managers

Define asynchronous context manager
```
class AsyncContextManager():
    async def __aenter__(self):
        await asyncio.sleep(1)
    def __aexit__(self, et, exc,
tb):
        await asyncio.sleep(1)
```

Use asynchronous context manager
```
async with CustomClass() as mgmr:
    # ...
```